

Pesquisa Linear ($O(n)$) vs Pesquisa Binária ($O(\log n)$)

Comparação de algoritmos de busca por contagem de passos e benchmark por tempo

Integrante

Wellington Torres

Atividade	Aula 03 - Atividade 01
Prazo de entrega	25 de agosto de 2026
Data de geração deste relatório	18/08/2026
Maior lista testada	1.000.000 elementos
Ambiente de execução	Python 3.14.3 (semente 42)
Tempo total do experimento	26,97 s

Relatório reproduzível: todos os números deste documento foram medidos pelo script atividade01.py e transcritos automaticamente por gerar_pdf.py.

1. Objetivo

Implementar o algoritmo de **pesquisa linear**, de complexidade $O(n)$, e compará-lo com a **pesquisa binária**, de complexidade $O(\log n)$, demonstrando a diferença entre os dois por duas evidências independentes: a **quantidade de passos** efetivamente executados e o **tempo de execução medido** (benchmark por tempo), usando listas grandes.

2. Metodologia

Cada algoritmo foi instrumentado para devolver, além do índice encontrado, o número de comparações que executou. Assim a contagem de passos não é uma estimativa teórica: é o valor real da execução.

- **Listas de teste:** 1.000, 10.000, 100.000, 1.000.000 elementos, ordenadas e sem repetições (`list(range(n))`) — a ordenação é pré-requisito da busca binária.
- **Cenários:** melhor caso (alvo na 1ª posição), pior caso com o alvo na última posição, pior caso com o alvo **ausente** da lista e caso médio (média de 25 alvos sorteados).
- **Medição de tempo:** `time.perf_counter`, relógio monotônico de alta resolução. Cada medição repete a chamada até esgotar um orçamento de 0,20 s e reporta a **mediana** das amostras — mais robusta a interferência do sistema operacional do que a média.
- **Reprodutibilidade:** semente fixa (`random.seed(42)`), de modo que uma nova execução sorteia exatamente os mesmos alvos.

Por que medir as duas coisas? A contagem de passos isola o algoritmo: é a mesma em qualquer máquina. O tempo mede o efeito real, mas depende de CPU, cache e da linguagem. Quando as duas evidências apontam na mesma direção, a conclusão não é um artefato da máquina de teste.

3. Implementação

3.1 Pesquisa linear — $O(n)$

Percorre a lista do início ao fim comparando elemento por elemento. Não exige lista ordenada e para assim que encontra o alvo.

```
def busca_linear(lista: list[int], alvo: int) -> tuple[int, int]:
    """Pesquisa linear -  $O(n)$ .

    Percorre a lista do inicio ao fim comparando elemento por elemento.
    Nao exige lista ordenada.

    Retorna (indice, passos); indice == -1 quando o alvo nao existe.
    """
    passos = 0
    for indice, valor in enumerate(lista):
        passos += 1          # 1 passo = 1 comparacao
        if valor == alvo:
            return indice, passos
    return -1, passos
```

3.2 Pesquisa binária — $O(\log n)$

Compara o alvo com o elemento do meio e descarta a metade que comprovadamente não pode conter o alvo, repetindo o processo sobre a metade restante: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$.

```
def busca_binaria(lista: list[int], alvo: int) -> tuple[int, int]:
    """Pesquisa binaria -  $O(\log n)$ .

    A cada passo descarta metade do espaco de busca. EXIGE lista ordenada,
    pois usa a comparacao com o elemento do meio para decidir qual metade
    ainda pode conter o alvo.

    Retorna (indice, passos); indice == -1 quando o alvo nao existe.
    """
    passos = 0
    baixo, alto = 0, len(lista) - 1
    while baixo <= alto:
        passos += 1          # 1 passo = 1 comparacao com o meio
        meio = (baixo + alto) // 2
        chute = lista[meio]
        if chute == alvo:
            return meio, passos
        if chute < alvo:
            baixo = meio + 1  # descarta a metade da esquerda
        else:
            alto = meio - 1   # descarta a metade da direita
    return -1, passos
```

4. Validação de corretude

Antes de comparar desempenho é preciso garantir que os dois algoritmos estão corretos — um algoritmo rápido e errado não serve de base de comparação. O script executa estas verificações a cada execução, e qualquer falha interrompe o experimento:

- 6 alvos-chave conferidos em lista de 1000 itens.
- 500 alvos aleatórios - linear e binária retornaram o mesmo índice em todos.
- passos da binária $\leq \lceil \log_2(n) \rceil + 1$ em todos os n testados (limite teórico respeitado).
- casos de borda (lista vazia e lista unitária).

A verificação do limite teórico é a mais relevante para esta atividade: ela confirma empiricamente que a busca binária nunca excede $\lceil \log_2(n) \rceil + 1$ comparações.

5. Resultado 1 — quantidade de passos

Comparações efetivamente contadas durante a execução (não estimadas). A coluna *Razão* mostra quantas vezes a linear faz mais passos que a binária.

Cenário	n	Linear (passos)	Binária (passos)	Razão
melhor caso (1º elemento)	1.000	1,0	9,0	0,11x
melhor caso (1º elemento)	10.000	1,0	13,0	0,08x
melhor caso (1º elemento)	100.000	1,0	16,0	0,06x
melhor caso (1º elemento)	1.000.000	1,0	19,0	0,05x
pior caso (último elemento)	1.000	1.000,0	10,0	100x
pior caso (último elemento)	10.000	10.000,0	14,0	714x
pior caso (último elemento)	100.000	100.000,0	17,0	5.882x
pior caso (último elemento)	1.000.000	1.000.000,0	20,0	50.000x
pior caso (alvo ausente)	1.000	1.000,0	9,0	111x
pior caso (alvo ausente)	10.000	10.000,0	13,0	769x
pior caso (alvo ausente)	100.000	100.000,0	16,0	6.250x
pior caso (alvo ausente)	1.000.000	1.000.000,0	19,0	52.632x
caso médio (aleatório)	1.000	468,3	9,3	50x
caso médio (aleatório)	10.000	4.648,9	12,2	382x
caso médio (aleatório)	100.000	45.625,4	15,2	3.002x
caso médio (aleatório)	1.000.000	542.017,9	19,0	28.587x

Nos três cenários em que o alvo não está no início, a razão cresce junto com n — é exatamente isso que separa $O(n)$ de $O(\log n)$. No melhor caso a razão é menor que 1: a busca linear encontra o alvo em 1 comparação, enquanto a binária ainda precisa afunilar até a primeira posição. Notação assintótica descreve o comportamento no crescimento, não garante vantagem em todo caso particular.

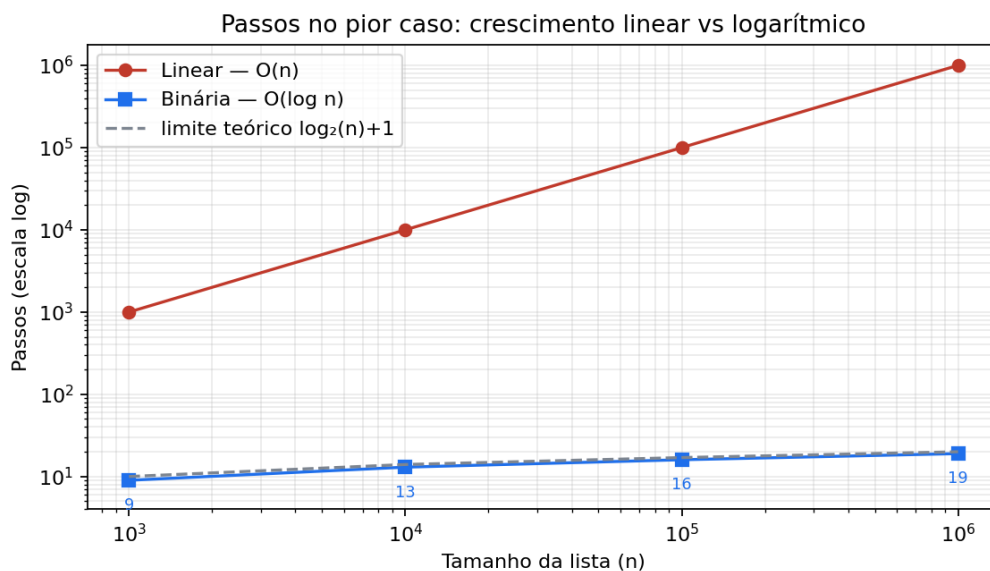


Figura 1 — Passos no pior caso (ambos os eixos em escala logarítmica). A curva da binária acompanha a reta teórica $\log_2(n)+1$; a da linear cresce na mesma proporção que n .

6. Resultado 2 — benchmark por tempo

Tempo mediano por busca, em microssegundos ($\mu\text{s} = 10^{-6} \text{ s}$).

Cenário	n	Linear (μs)	Binária (μs)	Speedup
melhor caso (1º elemento)	1.000	0,40	1,70	0,24x
melhor caso (1º elemento)	10.000	0,30	1,60	0,19x
melhor caso (1º elemento)	100.000	0,40	3,00	0,13x
melhor caso (1º elemento)	1.000.000	0,30	2,80	0,11x
pior caso (último elemento)	1.000	55,50	1,30	43x
pior caso (último elemento)	10.000	590,30	1,70	347x
pior caso (último elemento)	100.000	8.938,20	2,40	3.724x
pior caso (último elemento)	1.000.000	80.521,30	2,90	27.766x
pior caso (alvo ausente)	1.000	55,30	1,10	50x
pior caso (alvo ausente)	10.000	571,20	1,40	408x
pior caso (alvo ausente)	100.000	8.649,10	3,10	2.790x
pior caso (alvo ausente)	1.000.000	81.305,00	3,80	21.396x
caso médio (aleatório)	1.000	20,50	1,40	15x
caso médio (aleatório)	10.000	230,80	2,50	92x
caso médio (aleatório)	100.000	4.283,30	2,50	1.713x
caso médio (aleatório)	1.000.000	57.408,95	3,40	16.885x

No pior caso com 1.000.000 elementos, a busca linear levou **81.305,00 μs** (81,3 ms) contra **3,80 μs** da binária — cerca de **21.396x** mais rápida. O tempo confirma o que a contagem de passos previu, o que descarta a hipótese de a diferença ser um efeito da máquina de teste.

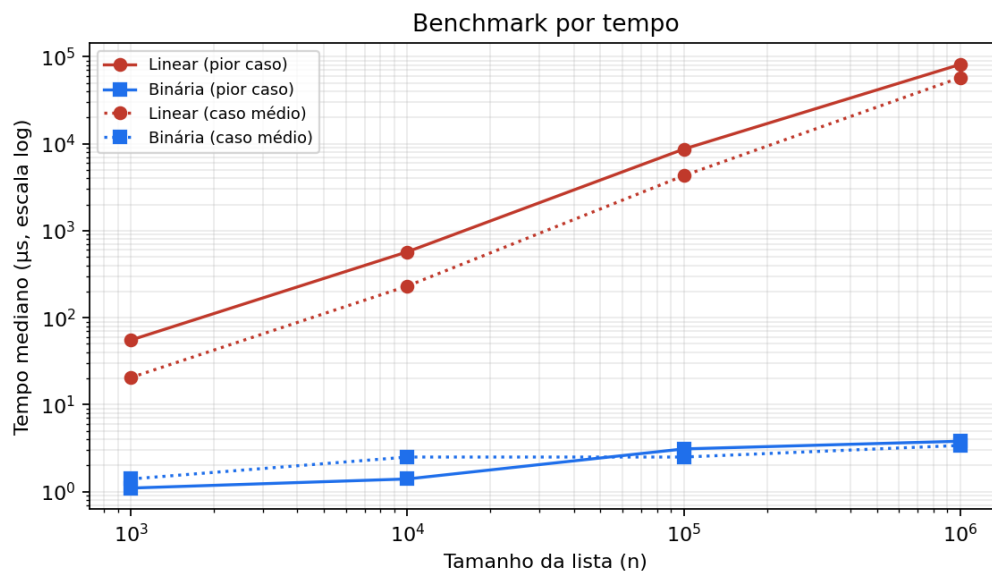


Figura 2 — Tempo mediano por busca. A linha da binária é praticamente horizontal: multiplicar a lista por 1.000 quase não altera seu tempo.

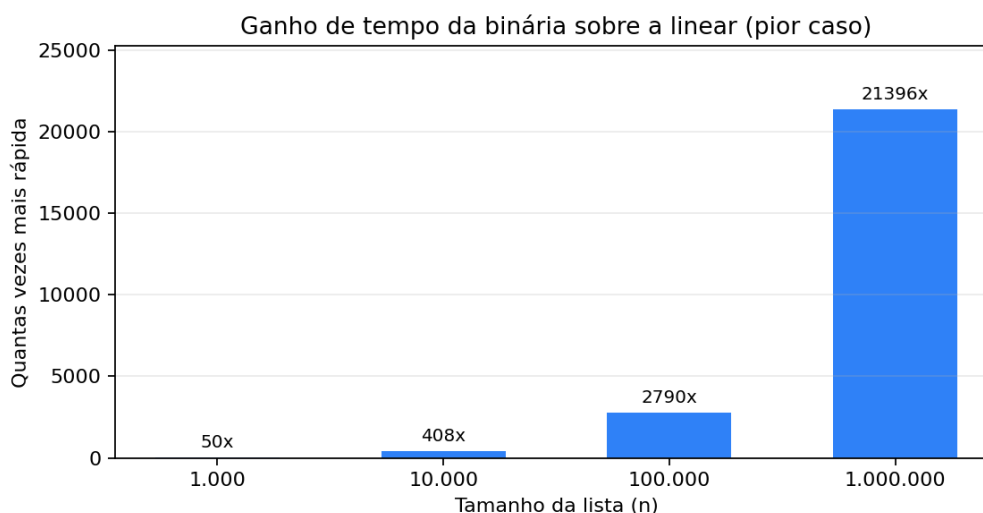


Figura 3 — Ganho de tempo da binária sobre a linear no pior caso. A vantagem não é constante: ela cresce com o tamanho da lista.

7. O preço do pré-requisito: ordenar a lista

A busca binária só funciona sobre lista ordenada, e ordenar não é grátis: custa $O(n \log n)$, **mais** que uma única busca linear $O(n)$. Medimos esse custo para responder à pergunta prática: a partir de quantas consultas vale a pena ordenar?

Operação (n = 1.000.000)	Tempo medido	Complexidade
Ordenar a lista (list.sort — Timsort)	451.750,60 μ s	$O(n \log n)$
1 busca linear (caso médio)	57.408,95 μ s	$O(n)$
1 busca binária (caso médio)	3,40 μ s	$O(\log n)$

O ponto de equilíbrio k sai da desigualdade “ k buscas lineares custam mais que ordenar + k buscas binárias”:

$$\begin{aligned}
 k * t_{\text{linear}} &> t_{\text{ordenacao}} + k * t_{\text{binaria}} \\
 \Rightarrow k &> t_{\text{ordenacao}} / (t_{\text{linear}} - t_{\text{binaria}})
 \end{aligned}$$

$k \approx 7,9$ consultas. Para menos de ~ 8 buscas em uma lista desordenada, a pesquisa linear é a escolha certa. A partir de ~ 8 consultas, ordenar uma única vez e usar a binária em todas as seguintes passa a ser mais barato — e a vantagem só aumenta daí em diante.

É por isso que a resposta “qual algoritmo é melhor?” depende do uso: busca única em dado desordenado favorece a linear; consultas repetidas na mesma base favorecem manter a base ordenada (ou indexada) e usar a binária.

8. Respostas às questões

8.1 Qual a diferença entre os dois algoritmos em termos de quantidade de passos no pior caso?

No pior caso — alvo na última posição ou **ausente** da lista — a pesquisa linear precisa comparar **todos** os elementos: são exatamente **n passos**. A pesquisa binária, no mesmo pior caso, executa no máximo **$\lfloor \log_2(n) \rfloor + 1$ passos**, porque cada comparação elimina metade do espaço de busca restante; o número de divisões por 2 necessárias para reduzir n a 1 é justamente $\log_2(n)$.

Nos dados medidos, com uma lista de 1.000.000 elementos: **1.000.000 passos** na linear contra **19 passos** na binária — **52.632x menos passos**, o que se traduziu em 81.305,00 µs contra 3,80 µs de tempo medido (21.396x mais rápida). O limite teórico para esse n é $\lfloor \log_2(1.000.000) \rfloor + 1 = 20$ passos, e a medição respeitou o limite.

Mais importante que o valor absoluto é **como cada um reage ao crescimento da lista**: dobrar n **dobra** o trabalho da busca linear, mas acrescenta **um único passo** à busca binária.

n	Passos O(n)	Passos O(log n)	Efeito de dobrar n
1.000	1.000	10	—
2.000	2.000	11	linear: x2 binária: +1 passo
4.000	4.000	12	linear: x2 binária: +1 passo
8.000	8.000	13	linear: x2 binária: +1 passo

8.2 Por que a busca binária exige que a lista esteja ordenada?

Porque a busca binária **não usa a comparação para encontrar** o elemento, e sim para **decidir qual metade da lista pode ser descartada**. Quando ela conclui “o alvo é maior que o elemento do meio, logo só pode estar à direita”, está afirmando algo sobre elementos que **nunca olhou**. Essa inferência só é válida se a ordenação garantir o invariante: todo elemento à esquerda do meio é menor ou igual a ele, e todo elemento à direita é maior ou igual.

Sem ordenação, o invariante quebra e o algoritmo descarta uma metade que podia conter o alvo. O efeito não é ficar mais lento — é ficar **incorreto**, devolvendo “não encontrado” para um elemento que existe (falso negativo), o que é bem pior que ser lento, porque falha em silêncio:

```
lista = [7, 2, 9, 1, 5]      # NAO ordenada
alvo = 1                    # existe, no indice 3

passo 1: meio = indice 2 -> valor 9 ; 1 < 9 -> descarta a DIREITA
          espaco restante = [7, 2]
passo 2: meio = indice 0 -> valor 7 ; 1 < 7 -> descarta a DIREITA
          espaco restante = [] -> retorna -1 (NAO ENCONTRADO)

Resultado ERRADO: o alvo estava na lista, no indice 3, mas foi
descartado no passo 1 junto com a metade da direita.
```

Vale registrar o outro lado dessa exigência: a ordenação tem um custo, medido na seção 7. Ordenar a lista de 1.000.000 elementos levou 451.750,60 µs — cerca de 7,9x o custo de uma única busca linear. Por isso a binária só compensa quando a lista já está ordenada ou quando o custo da ordenação é amortizado por muitas consultas (a partir de ~8 neste

experimento).

8.3 Escreva em suas palavras a diferença entre $O(n)$ e $O(\log n)$.

$O(n)$ e $O(\log n)$ não são algoritmos nem “tipos de busca”: são **taxas de crescimento** — descrevem como o custo de um algoritmo aumenta conforme o tamanho da entrada (n) aumenta. A busca linear é *um exemplo* de $O(n)$ e a binária é *um exemplo* de $O(\log n)$, mas a notação fala do comportamento, não da implementação.

- **$O(n)$ — crescimento linear:** o trabalho cresce na mesma proporção que a entrada. O algoritmo precisa, no pior caso, olhar cada elemento uma vez. Lista 10× maior, trabalho 10× maior. É o custo inevitável quando não se pode assumir nada sobre a organização dos dados.
- **$O(\log n)$ — crescimento logarítmico:** cada passo **elimina uma fração fixa** do problema (na binária, metade). O trabalho cresce com o logaritmo da entrada: lista 10× maior custa apenas ~3,3 passos a mais; lista 1.000× maior, ~10 passos a mais. Só é possível porque o algoritmo explora uma estrutura pré-existente nos dados — a ordenação.

A diferença fica evidente ao escalar n :

n	$O(n)$ — passos no pior caso	$O(\log n)$ — passos no pior caso
1.000	1.000	10
1.000.000	1.000.000	20
1.000.000.000	1.000.000.000	30

Em uma lista de 1 bilhão de elementos, o pior caso de $O(n)$ é 1 bilhão de comparações, enquanto o de $O(\log n)$ são 30. Ou, invertendo a pergunta: com o orçamento de 30 comparações, $O(\log n)$ cobre 1 bilhão de elementos e $O(n)$ cobre 30.

Duas observações que completam a resposta. **Primeira:** a notação Big O descreve o crescimento e ignora constantes, então ela não garante vantagem em *todo* caso — no melhor caso a busca linear acha o alvo em 1 passo, menos que a binária (seção 5). O que $O(\log n)$ garante é que a vantagem **aumenta** com n . **Segunda:** a base do logaritmo não aparece na notação porque mudar de base só multiplica o resultado por uma constante; escreve-se $O(\log n)$ mesmo quando o algoritmo divide o problema por 2, por 3 ou por 10.

9. Conclusão

- **Passos:** no pior caso, linear = n e binária = $\lfloor \log_2(n) \rfloor + 1$. Com $n = 1.000.000$, isso é 1.000.000 contra 20 comparações.
- **Tempo:** a medição confirmou a previsão — 81.305,00 μ s contra 3,80 μ s, 21.396x mais rápida no pior caso.
- **O ganho cresce com n :** a vantagem da binária não é um fator fixo; ela aumenta conforme a lista cresce (Figura 3). É essa propriedade que a notação assintótica captura.
- **Não é de graça:** a binária exige ordenação, que custa $O(n \log n)$. Abaixo de ~ 8 consultas sobre uma lista desordenada, a busca linear ainda é a escolha mais eficiente (seção 7).

Escolher entre $O(n)$ e $O(\log n)$, portanto, não é escolher o algoritmo “mais rápido” em abstrato: é verificar se os dados oferecem a estrutura (ordenação) que o algoritmo mais eficiente exige, e se o número de consultas justifica construir essa estrutura.

10. Requisitos da atividade

Requisito da atividade	Onde foi atendido
Implementar pesquisa linear $O(n)$	Seção 3.1
Comparar com pesquisa binária	Seções 3.2, 5 e 6
Provar a quantidade de passos	Seção 5 (contagem instrumentada)
Provar a eficiência em tempo (benchmark)	Seção 6 (mediana, perf_counter)
Usar uma lista grande	até 1.000.000 elementos
Responder às 3 questões	Seções 8.1, 8.2 e 8.3
Nome de todos no documento	Capa

Apêndice — como reproduzir

Os dois scripts estão no mesmo diretório deste PDF. Não há dependências além de matplotlib (gráficos) e reportlab (PDF):

```
python atividade01.py      # roda o experimento -> saida/resultados.json + graficos
python gerar_pdf.py        # monta este PDF a partir do JSON
```

A semente fixa garante que uma nova execução repita exatamente os mesmos alvos sorteados. Os tempos absolutos variam conforme a máquina, mas a razão entre os algoritmos e a forma das curvas se mantêm — é isso que a análise assintótica prevê.